HR ASSISTANT AI

Conversational AI HR Support Platform (RAG Architecture)

PRODUCT REQUIREMENTS & ARCHITECTURE DOCUMENT

Version 1.0 · 2026 · Yohannes Bayeh

Field	Detail
Project Name	HR Assistant AI
Type	Full-Stack Conversational AI Application — RAG Architecture
Developer	Yohannes Bayeh
Year	2025
Frontend URL	hr-assistant-o1ktjq8e8-john-bayehs-projects.vercel.app
Stack	React 19 + Vite · FastAPI · Groq (LLaMA 3.3 70B) · LangChain · ChromaDB
Theme	Dark UI — Blue accent (#2563EB) for user messages, dark card AI responses

SECTION 1 — EXECUTIVE SUMMARY, PROBLEM STATEMENT & SOLUTION

1.1 Executive Summary

HR Assistant AI is a conversational, AI-powered HR support tool that gives employees instant, natural-language answers to common human resources questions. It combines a Retrieval-Augmented Generation (RAG) pipeline with a large language model to ground responses in an organization's actual policy documents and employee records, reducing dependency on manual HR intervention for routine, repetitive queries.

The application is built as a full-stack system with a React-based single-page frontend and a Python FastAPI backend, deployed independently on Vercel and Render. It is currently positioned as a standalone technology demonstration, architected to be directly extensible toward integration with production HR systems such as AskualaLink in future iterations.

1.2 Problem Statement

Problem	Impact
Employees must search lengthy policy PDFs manually	Wastes employee time and reduces productivity
Routine questions are redirected to HR staff	Increases HR workload with repetitive, low-complexity requests
Policy information is inconsistently communicated	Leads to confusion, mistrust, or compliance risk
No instant access to personal/departmental HR data	Slows down basic verification tasks (e.g., checking performance scores)
Limited digitization of Ethiopian labor law guidance	Employees lack a quick reference for basic legal rights

1.3 Solution

- A conversational chatbot interface that understands natural-language HR questions.
- A retrieval layer (ChromaDB) that grounds AI answers in real policy content, reducing hallucination risk.
- A queryable employee dataset that answers structured questions about salaries, departments, and performance scores.

- Session-based conversational memory so employees can ask follow-up questions without repeating context.
- A zero-friction access model (no login required) for instant demonstration and testing.

SECTION 2 — CORE FEATURES & FUNCTIONAL REQUIREMENTS

2.1 Core Features

Feature	Description	Functional Requirement
AI HR Chatbot	Conversational interface for employees to ask HR-related questions in natural language	Must return contextually relevant, policy-grounded responses to free-text HR queries
Employee Data Querying	Query structured employee attributes — salary, department, performance scores	Must parse queries referencing employee names/attributes and return accurate matches
Company Policy Knowledge Base	Stores and serves policy content on leave, sick days, bonuses, remote work, overtime	Must retrieve the most relevant policy chunk(s) via vector similarity search
Session Conversation Memory	Retains conversational context across multiple turns within a session	Must inject prior conversation turns into each new model call within the session
Chat History Persistence	Preserves chat history across browser reloads	Must persist messages in browser localStorage and rehydrate UI on load
Ethiopian Labor Law Guidance	General, non-binding guidance on Ethiopian labor law basics	Must supplement policy answers with foundational labor law context where relevant
Minimal Chat UI	Single-column, distraction-free chat interface	Must render a responsive chat window with sticky input bar and message bubbles

2.2 Query Domains Supported

Domain	Example Query	Data Source
Employee Info	“What is the performance score for [employee]?”	Hardcoded employee dataset

Domain	Example Query	Data Source
Company Policy	“How many sick days am I entitled to?”	ChromaDB vector store (policy embeddings)
Labor Law Basics	“What does Ethiopian labor law say about overtime pay?”	LLM general knowledge + policy context
General HR Guidance	“How do I request remote work?”	ChromaDB + LLM synthesis

SECTION 3 — TECHNICAL ARCHITECTURE

3.1 Architecture Overview

HR Assistant AI follows a decoupled client-server architecture with a Retrieval-Augmented Generation (RAG) pipeline on the backend:

```
[React Frontend / Vercel]         |  HTTPS / REST (JSON)         v [FastAPI Backend / Render]         |-- LangChain Orchestration (ConversationBufferMemory + RAG chain)         |-- ChromaDB Vector Store (policy embeddings)         |-- Hardcoded Employee Dataset (8 fictional profiles)         |-- Groq API -- LLaMA 3.3 70B (LLM inference)
```

3.2 Technology Stack

Layer	Technology	Purpose
Frontend Framework	React 19	Component-based UI rendering
Frontend Build Tool	Vite	Fast dev server and optimized production bundling
Backend Framework	Python + FastAPI	REST API layer for chat, retrieval, and data queries
AI Inference Engine	Groq API (LLaMA 3.3 70B)	Low-latency LLM inference for chatbot responses
Orchestration / Memory	LangChain (ConversationBufferMemory)	Manages conversational context and RAG chaining
Vector Database	ChromaDB	Stores embedded policy chunks for semantic retrieval
Frontend Hosting	Vercel	Deployment of the React SPA
Backend Hosting	Render	Deployment of the FastAPI web service
Client-Side Storage	Browser localStorage	Persists chat history across sessions/reloads

3.3 AI & RAG Workflow

- User Input — employee submits a natural-language question via the chat interface.

- Intent Routing — backend determines whether the query relates to employee data, policy, labor law, or general guidance.
- Retrieval (RAG) — ChromaDB performs vector similarity search across embedded policy chunks.
- Memory Injection — LangChain ConversationBufferMemory supplies prior conversation turns.
- Prompt Construction — retrieved context, history, and query are composed into a structured prompt.
- LLM Inference — the prompt is sent to the Groq API (LLaMA 3.3 70B) for response generation.
- Response Delivery — the answer is rendered as a chat bubble and appended to localStorage.

SECTION 4 — DATA SCOPE, PRIVACY & STANDALONE ARCHITECTURE

4.1 Data Scope

HR Assistant AI, in its current release, operates as a standalone, self-contained application. It does not connect to any live, production HR system.

Data Element	Source	Status
Employee Profiles	Hardcoded dataset of 8 fictional employee records	Static, embedded in backend code
Company Policies	Sample policy documents (leave, sick days, bonuses, remote work, overtime)	Static, embedded and vectorized in ChromaDB
Production HR Data (AskualaLink)	N/A	Not connected — no live integration in this release

4.2 AskualaLink Connection Status

The application is architected to be extensible toward integration with a production HR platform (referred to as AskualaLink), but no such connection is active in the current build. All employee and policy data used for demonstration purposes is fictional and does not represent real personnel records.

4.3 Privacy Considerations

Consideration	Current State
Real employee data	Not used — all data is fictional / synthetic
Data transmission	Queries and responses transmitted via HTTPS between frontend and backend
Data storage	Chat history stored client-side only, in browser localStorage
Third-party data exposure	Query text is sent to the Groq API for inference processing
Compliance scope	Not certified for production use with real personal or payroll data

SECTION 5 — DESIGN SYSTEM & UI COMPONENTS

5.1 Design Philosophy

The interface follows a minimal, distraction-free, single-column chat layout optimized for quick Q&A interactions, using a dark theme for reduced visual fatigue and a modern, professional aesthetic.

5.2 UI Components

Component	Description	Styling Notes
Application Shell	Full-height, single-column layout with no sidebar	Dark background theme throughout
User Message Bubble	Displays employee-submitted questions	Blue background, right-aligned
AI Response Bubble	Displays chatbot-generated answers	Dark card background, left-aligned
Input Bar	Sticky textarea fixed to the bottom of the viewport	Persists on screen during scroll
Ask Button	Single primary action button to submit a query	Paired directly with the input textarea
Chat History View	Scrollable message thread	Rehydrated from localStorage on load

5.3 UX Principles

- Simplicity — no navigation clutter, sidebars, or multi-panel views; the interface is entirely conversation-focused.
- Continuity — persisted chat history ensures users do not lose context after a page reload.
- Responsiveness — sticky input positioning keeps the ask action accessible regardless of scroll position.
- Visual Contrast — distinct color treatment for user vs. AI messages improves conversational readability.

SECTION 6 — TESTING, LIMITATIONS & DEPLOYMENT

6.1 Deployment Configuration

Component	Platform	Notes
Frontend (React SPA)	Vercel	Deployed as a static single-page application
Backend (FastAPI)	Render	Deployed as a Python web service

6.2 Known Environment Limitation

The live backend deployment on Render currently runs a lightweight fallback API rather than the full RAG pipeline, due to Python 3.14 dependency constraints affecting LangChain and ChromaDB compatibility on Render's free tier.

Environment	RAG / LangChain / ChromaDB Status
Local Development	Fully operational — complete RAG pipeline with vector retrieval and conversational memory
Live Production (Render Free Tier)	Limited — fallback API active due to dependency/runtime constraints

6.3 Current Limitations

Limitation	Description
No authentication	Zero-Auth model — anyone with the URL can access the application
Static dataset	Employee data limited to 8 fictional profiles; not dynamically updated
Fallback backend on production	Full RAG capabilities not guaranteed on the live Render deployment
No production data integration	Not connected to AskualaLink or any live HR system
No server-side chat storage	Chat history exists only in the client browser via localStorage

6.4 Testing Considerations

Test Area	Approach	Result
Chatbot Response Accuracy	Manual validation against known policy content	✅ Pass
Session Memory	Verify follow-up questions retain prior context within a session	✅ Pass
Chat Persistence	Verify chat history survives browser reloads via localStorage	✅ Pass
Cross-Environment Consistency	Compare local (full RAG) vs. production (fallback) response quality	⚠️ Partial
API Availability	Health checks against the live Render backend endpoint	✅ Pass

6.5 Recommended Future Enhancements

- Upgrade the Render deployment tier or adjust dependency versions to restore full RAG functionality in production.
- Introduce authentication and role-based access control for enterprise deployment.
- Connect to a live HR data source (e.g., AskualaLink) with appropriate data governance controls.
- Add server-side chat history storage for multi-device continuity.
- Expand the policy knowledge base beyond sample documents to full organizational policy sets.

APPENDIX — DOCUMENT CONTROL

Version	Date	Author	Changes
1.0	July 27, 2026	Yohannes Bayeh	Initial documentation — complete